

SVG_VAE

:A Learned Representation for Scalable
Vector Graphics

STUDENT

Won Jiwoo

AUTHOR

Raphael Gontijo
Lopes etal

	pg.
I What is SVG?	3
II SVG_VAE	8
III Current Trends in SVG Generation Models	9

SVG란?

- 이미지가 아닌 2D 도형을 그리는 프로그램 코드
→ 도형 명령어 시퀀스 (언어 모델이 다루기 좋음!)
- 해상도 무관
- 이산 + 연속 혼합

① 이산 정보 (무엇을 그릴지)

명령 타입	의미	역할
moveTo	펜 위치 이동 (그리기 X)	새로운 획 시작
lineTo	직선 그리기	기본 획
cubicBezier	곡선 그리기	곡률·스타일 표현
EOS	경로 종료	시퀀스 끝

② 연속 정보 (어떻게 그릴지)

명령에 따라	연속 파라미터	의미
moveTo	(dx, dy)	시작 위치 이동
lineTo	(dx, dy)	직선 방향·길이
cubicBezier	(dx1, dy1)	제어점 1
	(dx2, dy2)	제어점 2
	(dx3, dy3)	끝점

ex) `<path d="m 20 80 c 40 -60 120 -60 160 0" />`

현재 위치 = (0, 0)

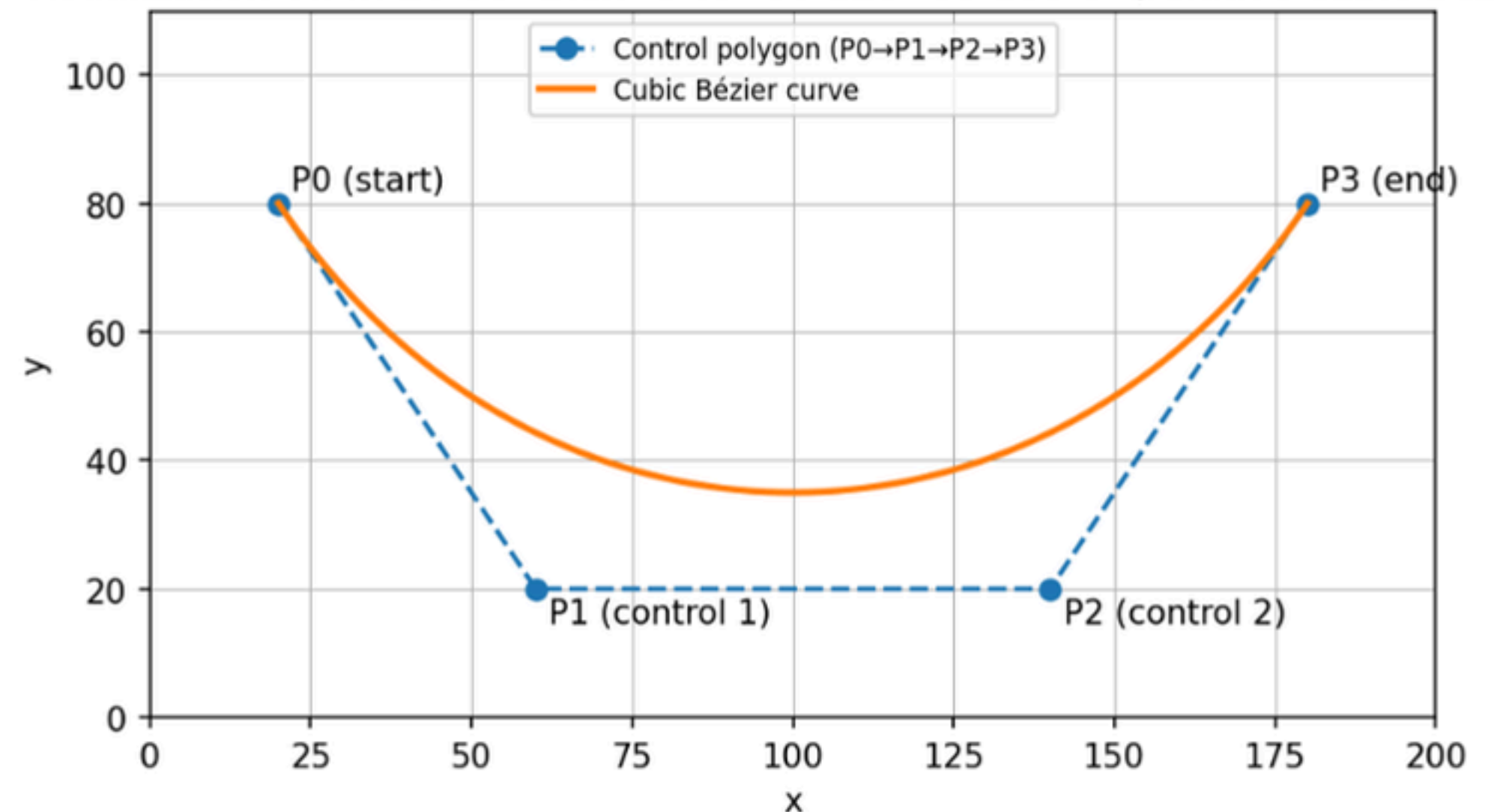
m 20 80

- (0,0) → (20,80) 선은 안 그림

c 40 -60 120 -60 160 0

- 제어점 1: $(20+40, 80-60) = (60,20)$
- 제어점 2: $(20+120, 80-60) = (140,20)$
- 끝점: $(20+160, 80+0) = (180,80)$

Visualization of: m 20 80 then c 40 -60 120 -60 160 0 (relative cubic Bézier)



ex) `<path d="m 10 10 l 90 0 l 0 90 l 10 10" />`

현재 위치 = (0, 0)

m 10 10

- (0,0) → (10,10) 선은 안 그림

l 90 0

- (10,10) → (100,10) 가로선 생성

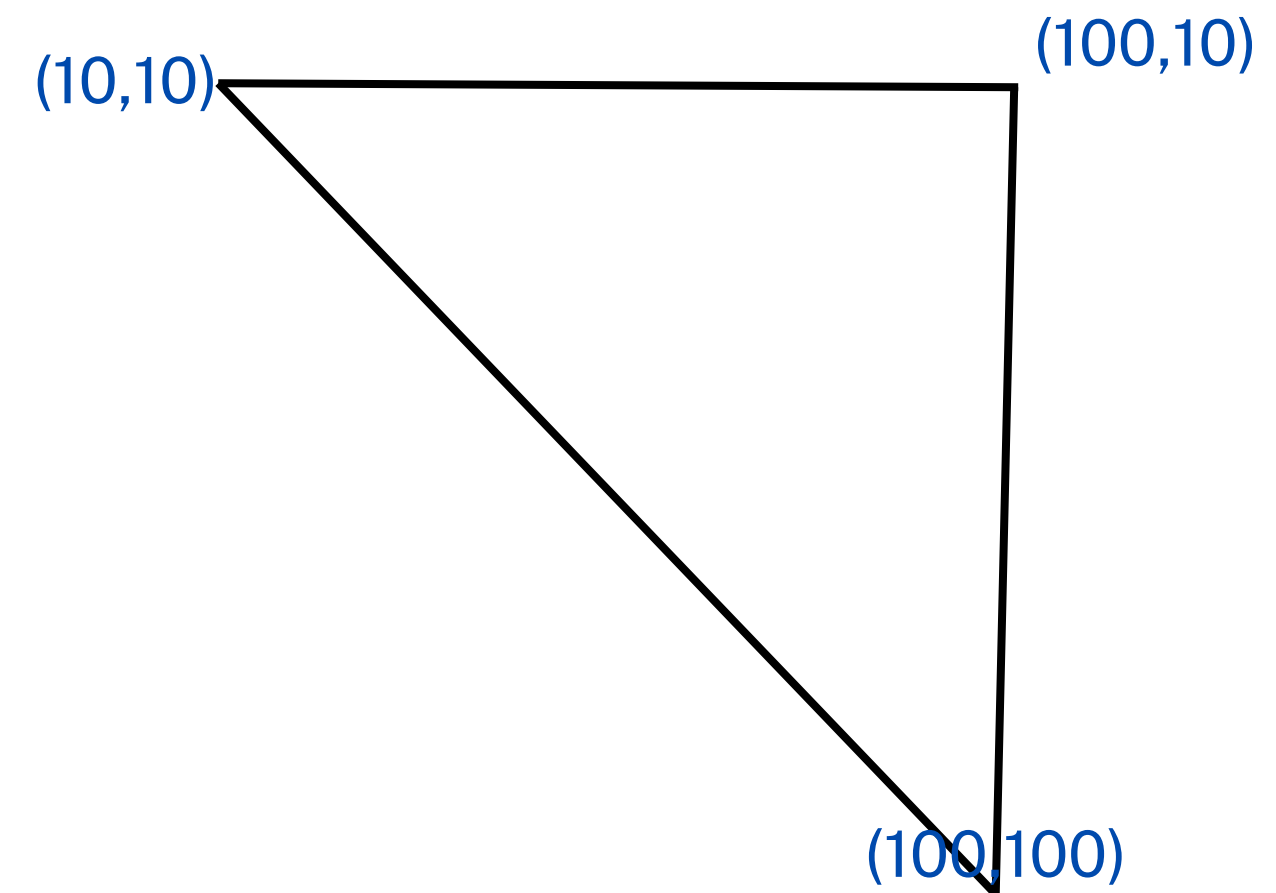
l 0 90

- (100,10) → (100,100) 세로선 생성

l 10 10

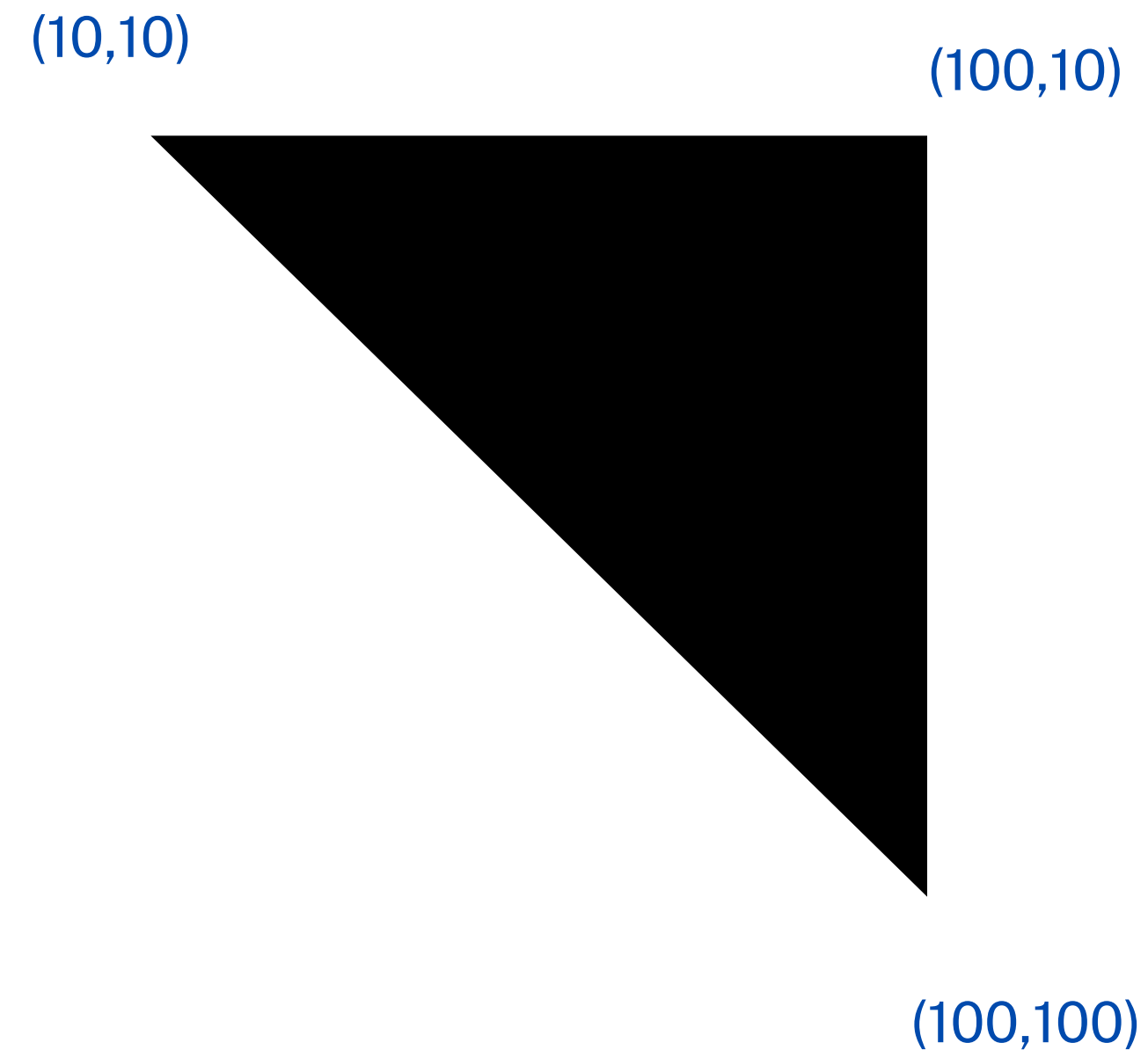
- (100,100) → (10,10) 직선 연결

=> 닫힌 도형이 됨 !



ex) 렌더링 설정

```
<svg viewBox="0 0 110 110"  
xmlns="http://www.w3.org/2000/svg">  
  <path d="m 10 10 l 90 0 l 0 90 l -90 -90"  
fill="currentColor"/>  
</svg>
```



SVG decoder input 형태

$x_t = \text{concat}(\text{prev_token}, \text{class_onehot}, z)$

1. 이전 step(t-1)의 출력
2. 현재 생성중인 문자 라벨
3. 스타일 잠재 벡터

z =
 [0.12, -0.45, 0.03, 1.10,
 -0.78, 0.66, -0.21, 0.04,
 0.91, -0.32, 0.15, -0.08,
 0.27, 0.44, -0.59, 0.02,
 0.11, -0.09, 0.38, -0.00...]

ex)

(1) 명령 타입 one-hot (Discrete)

- 명령 집합 순서 : [moveTo, lineTo, cubicBezier, EOS]
 $\text{cmd}_{\{t-1\}} = \text{lineTo} \rightarrow \text{one-hot} = [0, 1, 0, 0]$

(2) 연속 파라미터 (Continuous)

- 모든 명령을 최대 길이 6으로 통일 (padding + mask)

$\text{args}_{\{t-1\}} = (dx, dy) = (90, 0)$

$\rightarrow \text{padded args (6-dim)}: [90, 0, 0, 0, 0, 0]$

$\Rightarrow \text{prev_token} = [0, 1, 0, 0 \mid 90, 0, 0, 0, 0, 0]$

ex) label 수 62개,

class_onehot (62-dim): [0, 0, ..., 0, 1, 0, ..., 0]

↑

index 36

	pg.
I What is SVG?	3
II SVG_VAE	8
III Current Trends in SVG Generation Models	9

abstract

- 생성 모델의 진보 속에 더 높은 수준의 vision, image이해를 하려는 시도는 없었음
- 우리는 font drawing process를 모델링 함

→ scale-invariant representation

→ style propagatrion 수행 위해 조작 가능한 latent representation

→ font design 용이하게 하는 tool로 이용 가능

Introduction

“어떻게 *raw visual information*을 인간이 해석할 수 있는가?”

→ ‘Structured representations’이용!

- generalization과 efficient learning을 도움
- 탐색 가능한 building block을 제공
- 고수준의 semantics을 포착하는 간결함 강조→ 가능한 적은 그래픽 요소를 사용

Introduction

our goal : ‘많은 image set (시각적 인식) → Structured representaion(그리는 행동)’ drawing model을 훈련시키기

1. image내 기저하는 structure을 학습
2. 학습된 representation 기반으로 drawing을 생성

문제 : “inverse graphics problem”

- output representation이 pixel이 아니라 a sequence of discrete instruction임
- 1. latent variable에 대한 discrete representations를 학습해야 함
- 2. non-differenrial graphics engine을 이용해 최적화를 수행해야 함

previous approach

- program synthesis approach
- reinforcement와 adversarial learning을 활용

→ 일반화, 안전성, 확장성이 떨어짐

Introduction

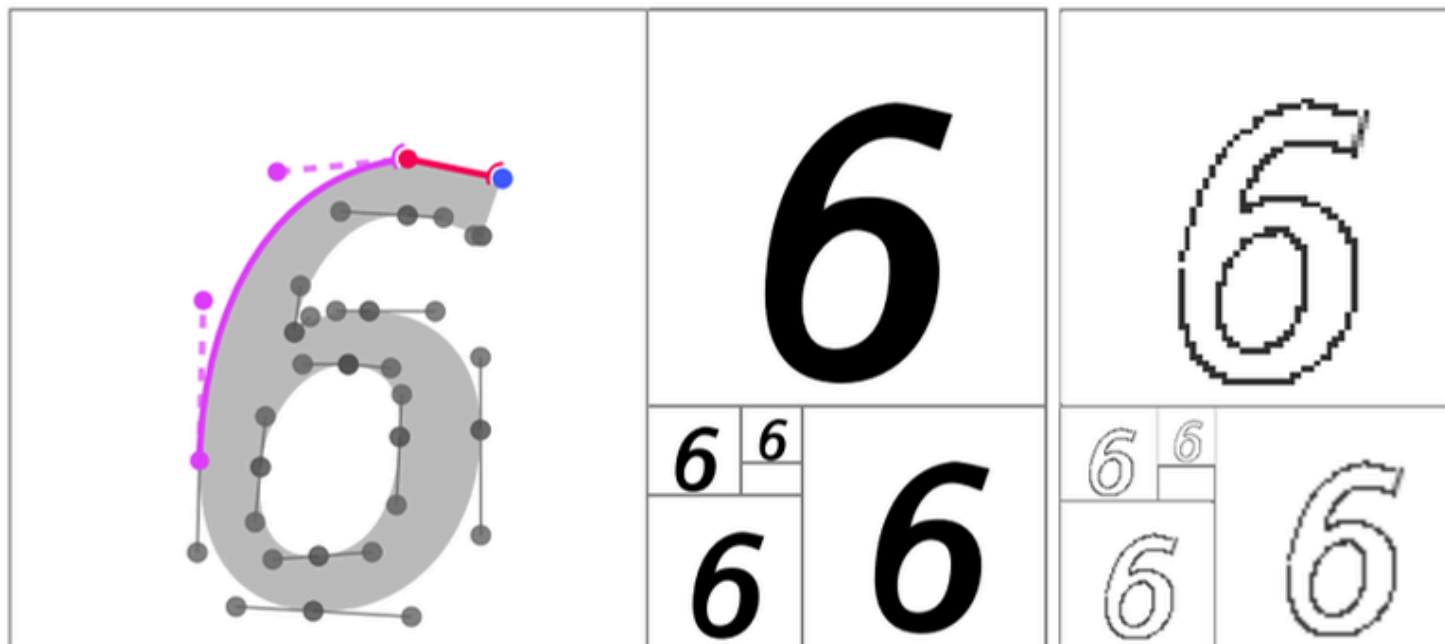
our approach

generating fonts specified with Scalable Vector Graphics(SVG)문제에 집중

SVG

- compact, scale-invariant representation
- illustration → a sequence of a higher level ‘command + numerical argument’

moveTo (15, 25)
lineTo (-2, 0.3)
cubicBezier (-7.4, 0.2) (-14.5, 11.7), (-12.1, 23.4)
...



Introduction

목표 : font의 character와 style에 대한 representation 학습 tool 만들기

our main contribution

1. SVG 생성 모델을 만들어 a large- scal dataset of 14M font charactor에 적용
2. font style에 대한 perceptually smooth latent representation을 제공
→ 다양한 폰트 양식 + 각 문자에 대해서는 일관됨
3. a single(or multiple) character의 잠재표현 이용 → complete SVG fontset를 추론
4. 잠재 표현 내 semantically meaningful direction을 확인 → 폰트 스타일 조작 가능!

Related work

generative models of images

- generative adversarial network
- probabilistic model

→ 다루기 쉬움

→ 훈련 불안정의 위험X

→ log-likelihood로 모델 quality 측정 가능

autoregressive generative models

- log-likelihood와 같은 unsupervised objective를 사용하는 생성모델 quality 향상 위해
- joint prediction → conditional, sequential prediction

- sequential 모델로 autoregressive-fashion에서 훈련

→ teacher-forcing training strategy로 훈련 (학습 안정)

→ 복잡분포 표현 위해 more sophisticated density modeling과 함께 사용됨 (MDN)

Related work

modeling higher level languages

- 프로그램 자체를 추론

((given graphic engine → program))

→ 미분불가엔진이므로 reinforce 알고리즘 사용

→ 그리는 행동이 계속 비슷해짐

→ 낮은 해상도에선 괜찮으나 SVG에서는 구조적 오류 발생

Learning representations for fonts

previous work

- single character의 고수준 feature(style + class)로부터 다른 class들 사이의 style propagation을 가능하게 함
→ 문자 골격등의 단순 feature, 유연성 부족
- 생성된 모든 character의 style manipulation을 다룸
→ weight, width등의 파라미터 조절 제한적
- 폰트 스타일의 manifold를 학습 (semantically-meaningful 잠재 공간을 학습)
- 각 문자의 outline을 따라 이산화된 points사이의 관계를 포착하는 에너지 모델을 학습(font 생성, 구간추정)
→ 유연성이 떨어짐, 더 큰 spatial scale에서 오류 결함 발생

Methods

Data

SFD형식의 14M examples across 62 characters (0~9, a~z, A~Z) 사용

SFD → SVG one-to-one mapping 진행

- 4 SVG commands (moveTo, lineTo, cubicBezier, EOS)
- top-most command부터 시계 방향 순서로 정규화 됨
- 총 명령 개수 50개 이하
- 상대 좌표 이용 (실험 결과 절대 좌표보다 좋았음)

최종 데이터 셋 : sequence of commands specified in tuples

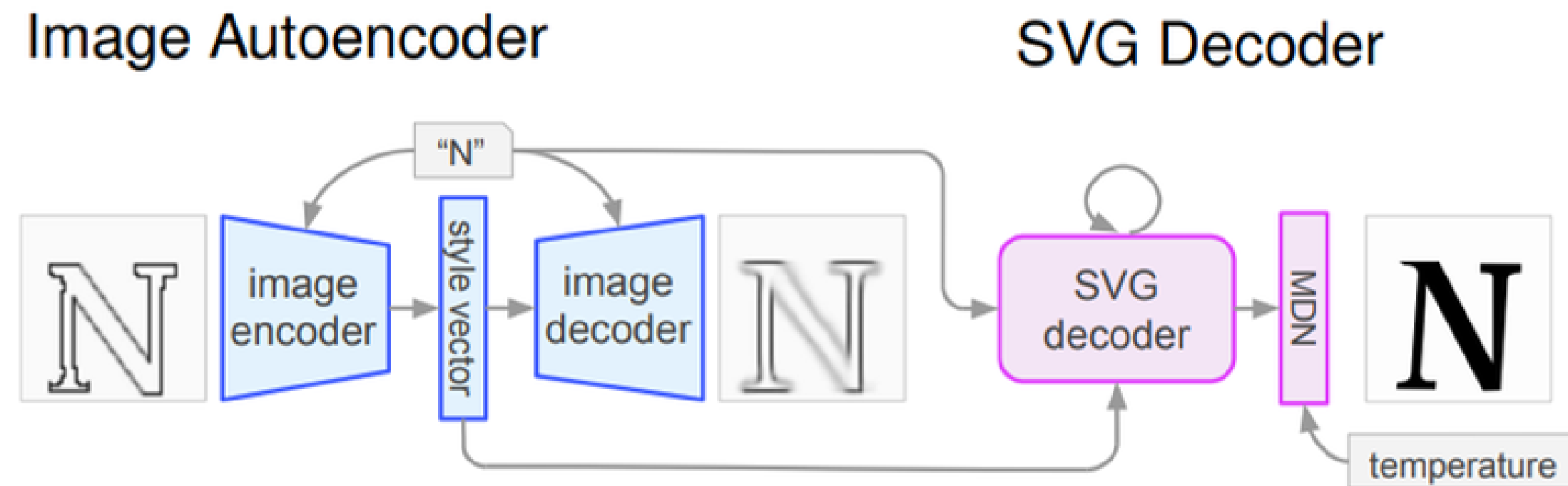
- SVG command + set of normalized floating-point numbers specifying command argument

한 문자 → 하나의 single path로 구성

- 학습 데이터 셋 12.6M + test 데이터 셋 1.4M
- SVG path → neural network 훈련에 적합한 vector format
→ (command type의 one-hot encoding + a normalized representation of command's argument)

Methods

Network architecture



variational autoencoder(CNN기반) + autoregressive SVG decoder(Tensor2Tensor 사용)

2.SVG_VAE

Methods

Network architecture

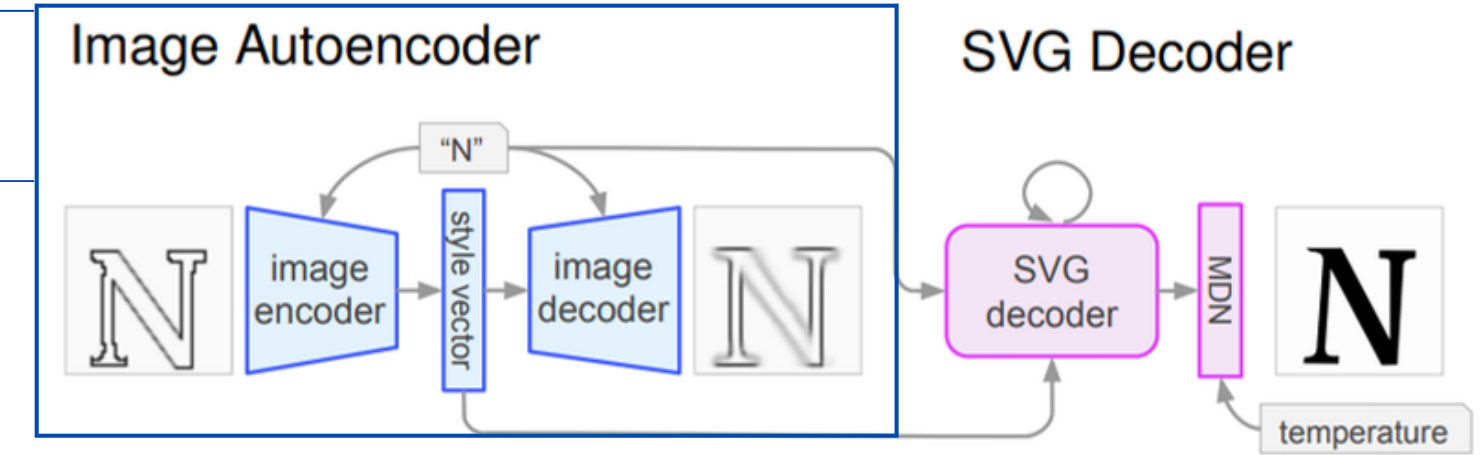
VAE = convolutional encoder /decoder + instance normalization layer(label에 따라 정규화 방식이 다름) + ReLU

class-conditioned autoencoder → class-independent latent code z (font 스타일에 대한 특징만 압축)

- 실험 결과 32 차원이 expressivity, tractability 사이의 balance를 잘 맞췄음
- train시 z 를 reparameterization trick으로 샘플링 (다양, 랜덤선택)
- test시 $z=\mu$ 를 사용 (결과 안정성)

Operations	Kernel, Stride	Output Dim
Conv-CIN-ReLU	5, 1	64x64x32
Conv-CIN-ReLU	5, 1	32x32x32
Conv-CIN-ReLU	5, 1	32x32x64
Conv-CIN-ReLU	5, 2	16x16x64
Conv-CIN-ReLU	4, 2	8x8x64
Conv-CIN-ReLU	4, 2	4x4x64
Flatten-Dense	-	64

Table 1: Architecture of convolutional image encoder containing 416, 672 parameters.



Operations	Kernel, Stride	Output Dim
Dense-Reshape	-	4x4x64
ConvT-CIN-ReLU	4, 2	8x8x64
ConvT-CIN-ReLU	4, 2	16x16x64
ConvT-CIN-ReLU	5, 1	16x16x64
ConvT-CIN-ReLU	5, 2	32x32x64
ConvT-CIN-ReLU	5, 1	32x32x32
ConvT-CIN-ReLU	5, 2	64x64x32
ConvT-CIN-ReLU	5, 1	64x64x32
Conv-Sigmoid	5, 1	64x64x1

Table 2: Architecture of convolutional image decoder containing 516, 865 parameters.

2.SVG_VAE

Methods

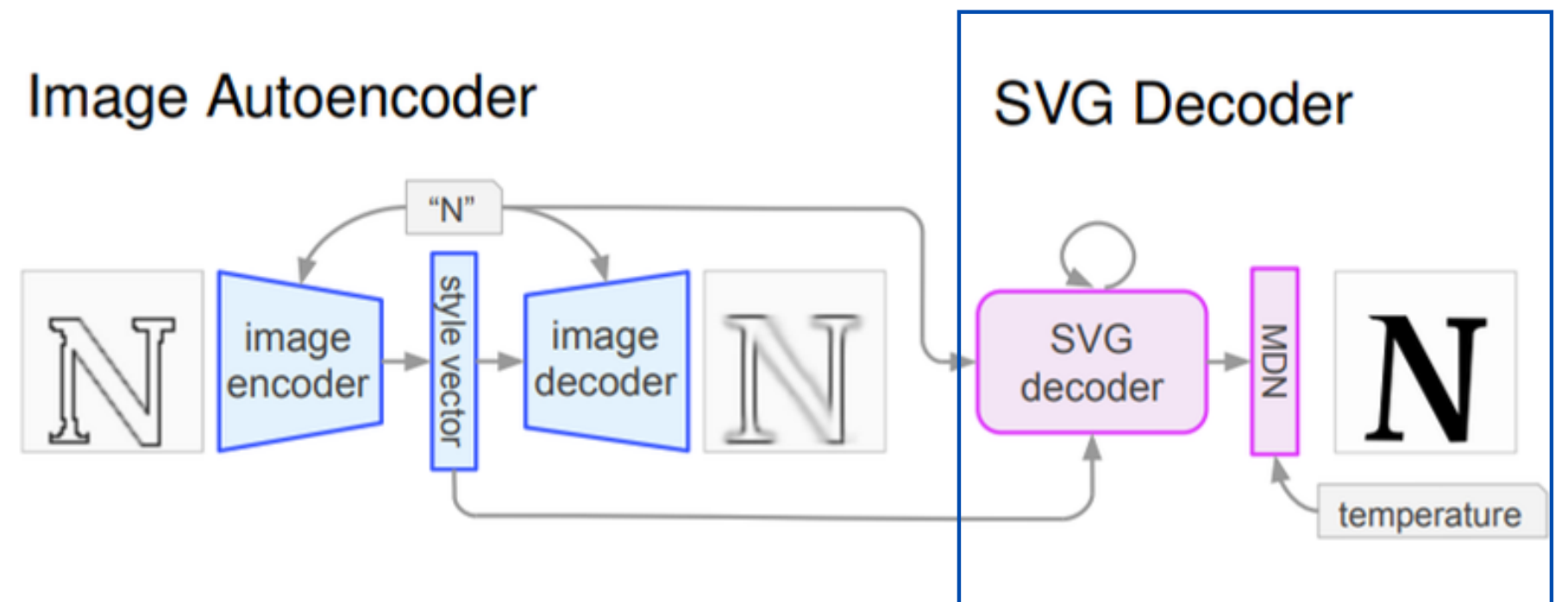
Network architecture

SVG decoder = 4 stacked LSTM + Mixture Density Network(MDN)

- feed forward dropout으로 훈련 (70% keep)
- LSTM의 input : previous sampled MDN output + class label + latent style representation z
- SVG decoder의 loss = softmax cross-entropy loss(one-hot SVG command) + MDN loss(real-valued argument)
- hidden state $\rightarrow$ 항상 z 에 의해 conditioned
- total number of parameter : 34,875,272

*SVG 명령 = 명령 type(이산적) + 좌표 인자(연속적)

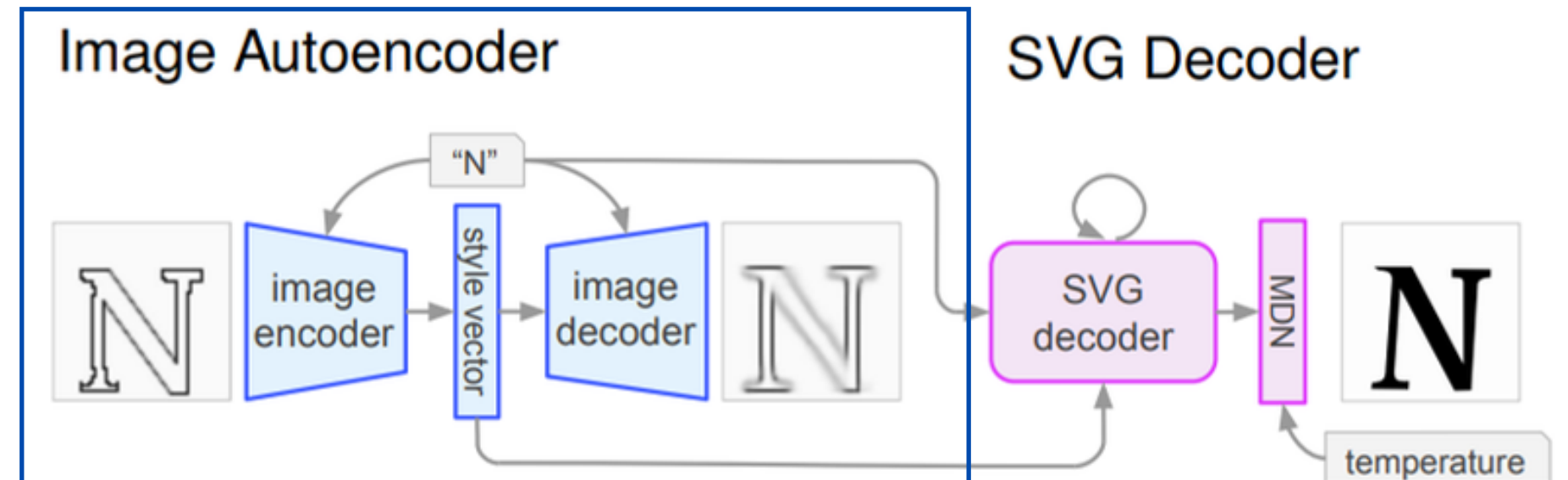
- 이산적 $\rightarrow$ soft max, 연속적 $\rightarrow$ MDN 사용
- MDN에서 각 명령어 type별 좌표 확률 분포를 예측 $\rightarrow$ loss 계산 시에는 정답 command의 좌표 확률 분포의 loss만 반영



Methods

training

=> VAE와 SVG decoder 각각 따로 훈련



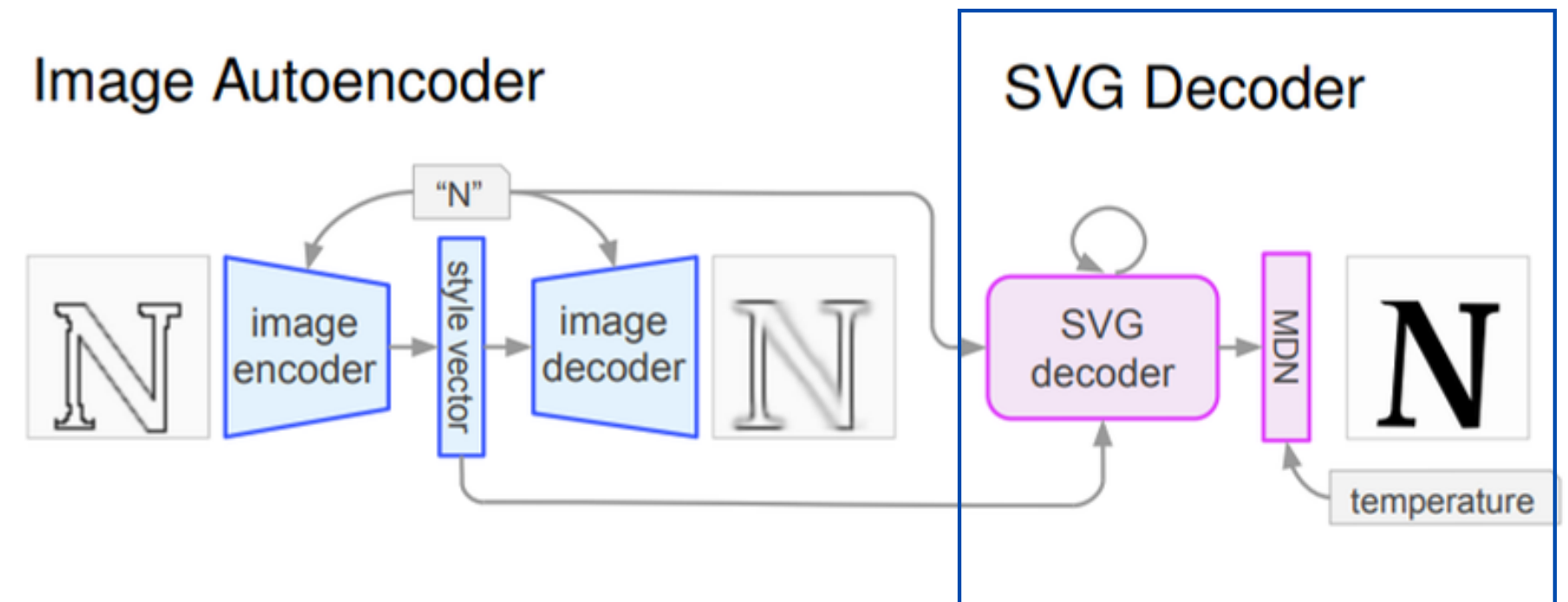
VAE

- log likelihood reconstruction loss(입력이미지를 얼마나 잘복원) + KL loss(잠재공간을 정규화)
- fonts pixel renderings, Adam optimizer($\epsilon = 10^{-6}$, 3 epoch) 사용 훈련
- KL β (정보 압축 강도) = 4.68 $\rightarrow$ 높게 설정 $\rightarrow$ 정보량을 강하게 제한 $\rightarrow$ 스타일을 잘 분리
- 문제 : z가 의미 없어짐 $\rightarrow$ free bit 조정(최소 정보량)
- 수렴 후 VAE는 frozen

Methods

training

=> VAE와 SVG decoder 각각 따로 훈련



SVG decoder

- softmax cross-entropy loss(one-hot command-type encoding 사이) + MDN loss(real valued argument)

*softmax cross entropy loss에 10을 곱함 (상대적으로 작은 값이므로 loss간 균형을 맞춤)

- teacher forcing 사용해 훈련 (이전 명령의 정답을 입력으로 넣음)
- Adam optimizer($\epsilon = 10^{-6}$, 3 epoch)

SVG Decoder in/output

input

$x_t = \text{concat}(\text{prev_token}, \text{class_onehot}, z)$

output $\rightarrow \text{output}_t = (\text{cmd}_t, \text{args}_t)$

$\text{cmd_logits} = [-1.32, 2.45, 0.61, -3.10]$ //moveTo/ lineTo /cubicBezier /EOS

$\text{pi_logits} = [0.84, 0.12, -0.53, 0.67, -1.10]$ //MDN mixture 가중치

$\mu = [[9.1, 0.2, 0.0, 0.0, 0.0, 0.0], \text{ // k1 } [8.7, -0.1, 0.0, 0.0, 0.0, 0.0], \text{ // k2 ... ~ k5}]$

$\text{log_sigma} = [[-0.28, -1.21, -2.0, -2.0, -2.0, -2.0], [-0.11, -1.52, -2.0, -2.0, -2.0, -2.0], \dots]$

command 샘플링

$\text{softmax}(\text{cmd_logits}) \approx [0.02, 0.79, 0.17, 0.01] \rightarrow \text{cmd}_t = \text{lineTo}$

좌표 샘플링 (*mixture k=4* 선택됨)

$\mu_4 = [8.5, 0.0, 0.0, 0.0, 0.0]$

$\text{log_sigma}_4 = [-0.19, -1.36, -2, -2, -2, -2]$

$\text{args_sample} = \mu_4 + \text{sigma}_4 \odot \varepsilon \ (\varepsilon \sim N(0, I))$

$\text{args}_t = (8.83, -0.31)$

token sequence → SVG path

```
glyph_tokens = [
  (moveTo, (100, 20)),
  (cubicBezier, (40, 0, 60, 40, 60, 80)),
  (cubicBezier, (0, 40, -40, 60, -80, 60)),
  (cubicBezier, (-40, 0, -60, -40, -60, -80)),
  (cubicBezier, (0, -40, 40, -60, 80, -60)),
  (moveTo, (100, 70)),
  (cubicBezier, (20, 0, 30, 20, 30, 40)),
  (cubicBezier, (0, 20, -20, 30, -40, 30)),
  (cubicBezier, (-20, 0, -30, -20, -30, -40)),
  (cubicBezier, (0, -20, 20, -30, 40, -30)),
  (EOS, ())
]
```



```
<path
  d="
    m 100 20
    c 40 0 60 40 60 80
    c 0 40 -40 60 -80 60
    c -40 0 -60 -40 -60 -80
    c 0 -40 40 -60 80 -60
    m 100 70
    c 20 0 30 20 30 40
    c 0 20 -20 30 -40 30
    c -20 0 -30 -20 -30 -40
    c 0 -20 20 -30 40 -30
  "
  fill="black"
/>
```

Results

- 14M examples의 데이터셋에 적용
- VAE + SVG decoder로 3 epoch 훈련됨



Figure 3: **Selected examples of generated fonts.** Examples generated by sampling a random latent representation z and running the SVG decoder by conditioning on z and all class labels. Each font character is selected as the best of 10 samples. See Figures 10 and 11 in Appendix for additional examples.

Analysis of representational ability

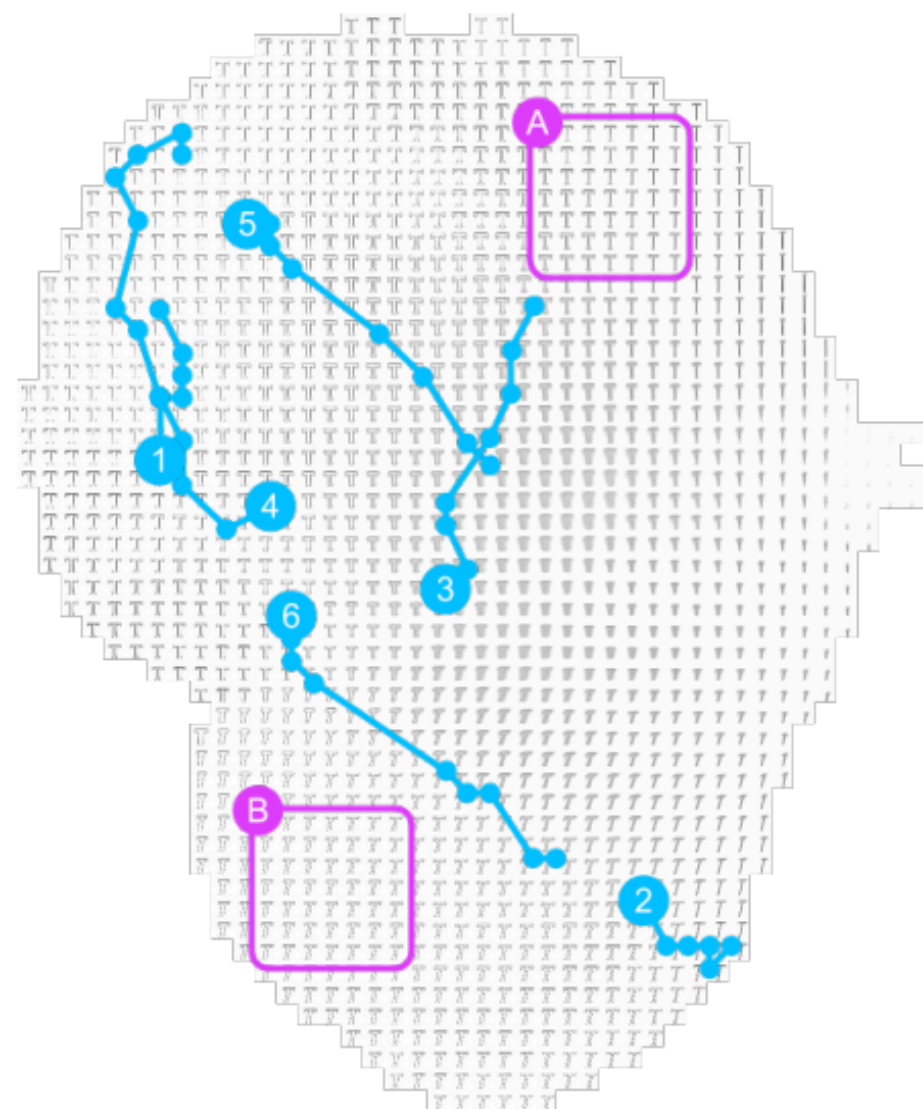
Learning a **smooth**, latent representation of font style

Q. 인지적으로 smooth하고 해석가능한 latent representation for font style이 학습 가능한가?

→ 1M 훈련예제에 대한 32-dimensional font-style → UMAP 이용 2차원으로 축소 → 이산화

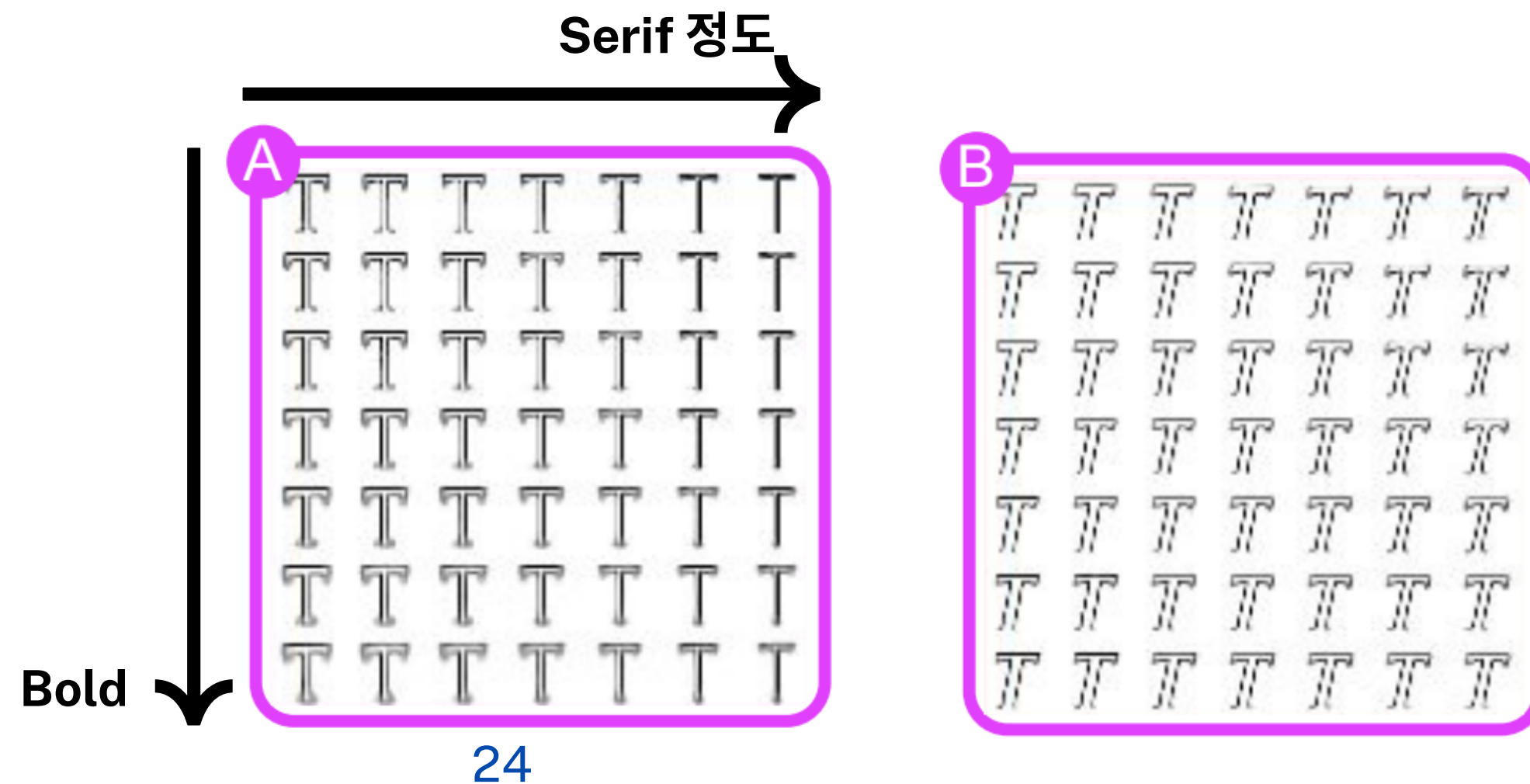
→ 각 grid 위치에서의 평균 z 를 픽셀기반 시각화

Learned Latent Space



→ manifold의 각 구역에서 문자의 smooth transition을 확인할 수 있음

→ local direction에 해당하는 visual semantic도 드러남



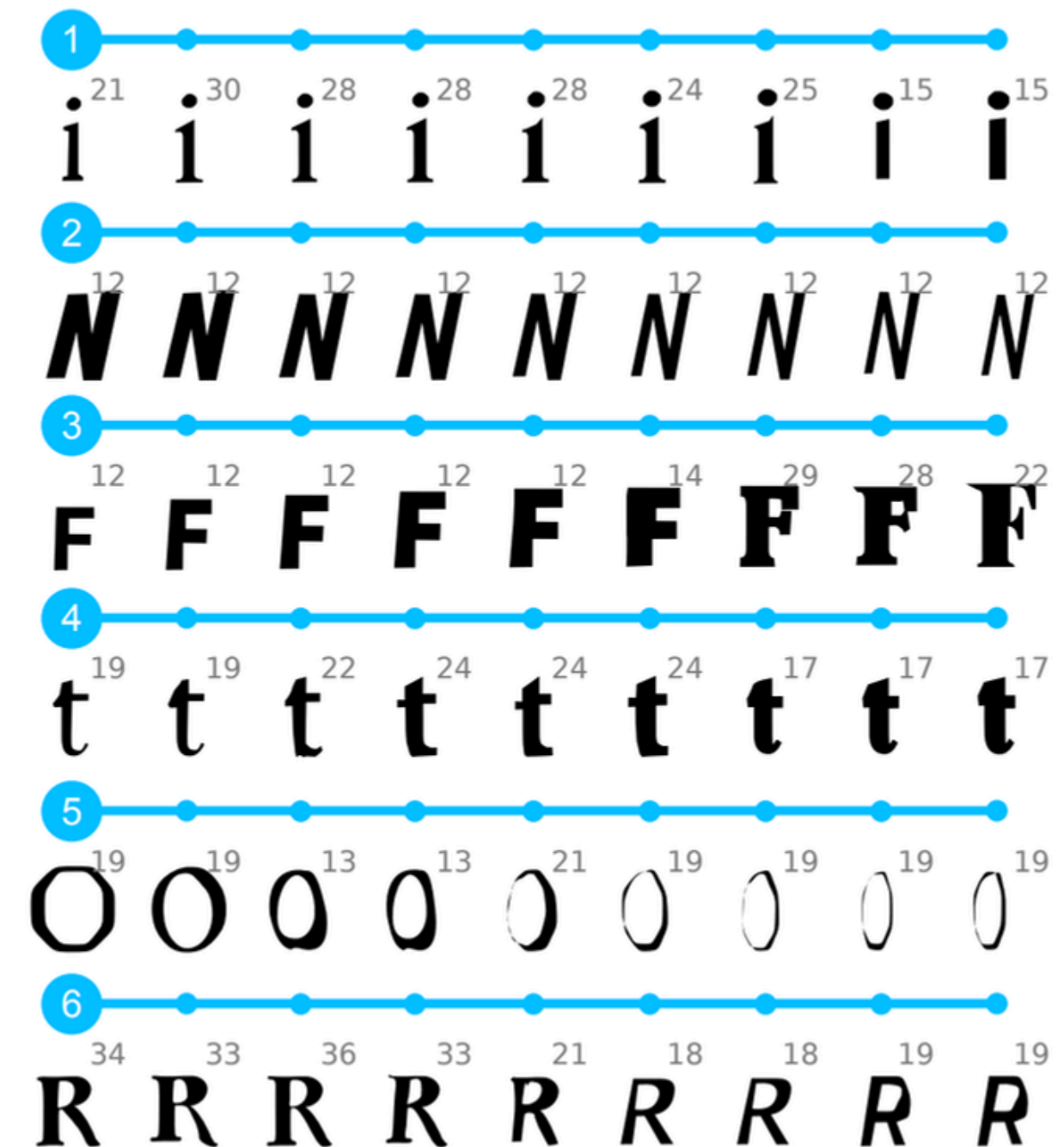
Analysis of representational ability

Learning a **smooth**, latent representation of font style

Q. 인지적으로 smooth하고 해석가능한 latent representation for font style이 학습 가능한가?

- 각 SVG쌍에 대한 $z \rightarrow$ linear interpolation 시각화
- decoded SVGs간의 smooth transition
- 수십 개의 이산 명령이 시각적으로 부드럽게 변하고 있는 것!

Linear Interpolation Between Two Characters



Analysis of representational ability

Exploiting the latent representation for **style propagation**

Q. 한 font set에서의 single character로부터 나머지 font set 추론 가능?

→ single character에 대한 잠재 표현 z 를 계산, SVG decoder에 z + 각 문자 label을 조건화함

0 1 2 3 4 5 6 7 8 9 A B C D E F

0 1 2 3 4 5 6 7 8 9 A B C D E F

0 1 2 3 4 5 6 7 8 9 A B C D E F

0 1 2 3 4 5 6 7 8 9 A B C D E F

0 1 2 3 4 5 6 7 8 9 A B C D E F

0 1 2 3 4 5 6 7 8 9 A B C D E F

0 1 2 3 4 5 6 7 8 9 A B C D E F

0 1 2 3 4 5 6 7 8 9 A B C D E F

0 1 2 3 4 5 6 7 8 9 A B C D E F

0 1 2 3 4 5 6 7 8 9 A B C D E F

0 1 2 3 4 5 6 7 8 9 A B C D E F

0 1 2 3 4 5 6 7 8 9 A B C D E F

- 각 row에 빨간 박스 문자의 z 로 조건화되어 나머지 문자들 생성됨
→ 각 row 일관되게 perceptually-similar style 지님

- 훈련시 latent space에서의 같은 point가 perceptually similar 스타일을 갖도록 강제하지 않았음에도 unsupervised manner로 class label너머의 일관성이 학습됨!

=> single value of z 는 perceptually similar set of character과 상응함

- font style 다양함
→ latent space z 는 많은 스타일 다양성 포착, 학습 가능

- 각 열은 같은 class에 속함
→ z 는 style 정보를 다른 문자 label 너머 일관되게 인코딩함
→ 모델은 class label과 style을 분리하여 처리함을 알 수 있음

Analysis of representational ability

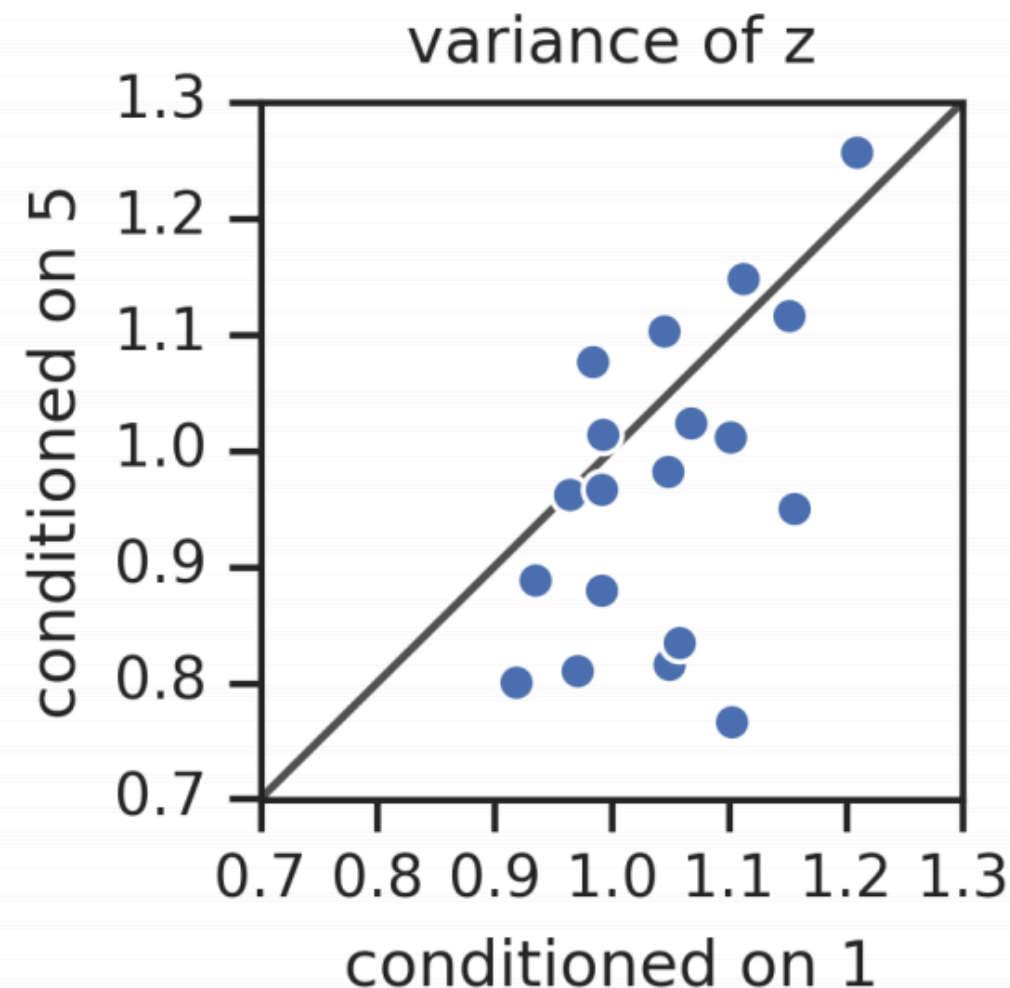
Exploiting the latent representation for **style propagation**

Q. 더 많은 문자를 활용한다면 style propagation의 성능을 더 향상시킬 수 있는가?

→ multiple character에 대한 잠재 표현 z 를 계산, SVG decoder에 평균 z + 각 문자 label을 조건화함

0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F
 0 1 2 3 4 5 6 7 8 9 A B C D E F

- style consistency와 individual outputs의 quality가 더 향상됨
- style propagation이 self-consistent 했다면 각 문자의 z 를 측정시 모두 동일 해야함



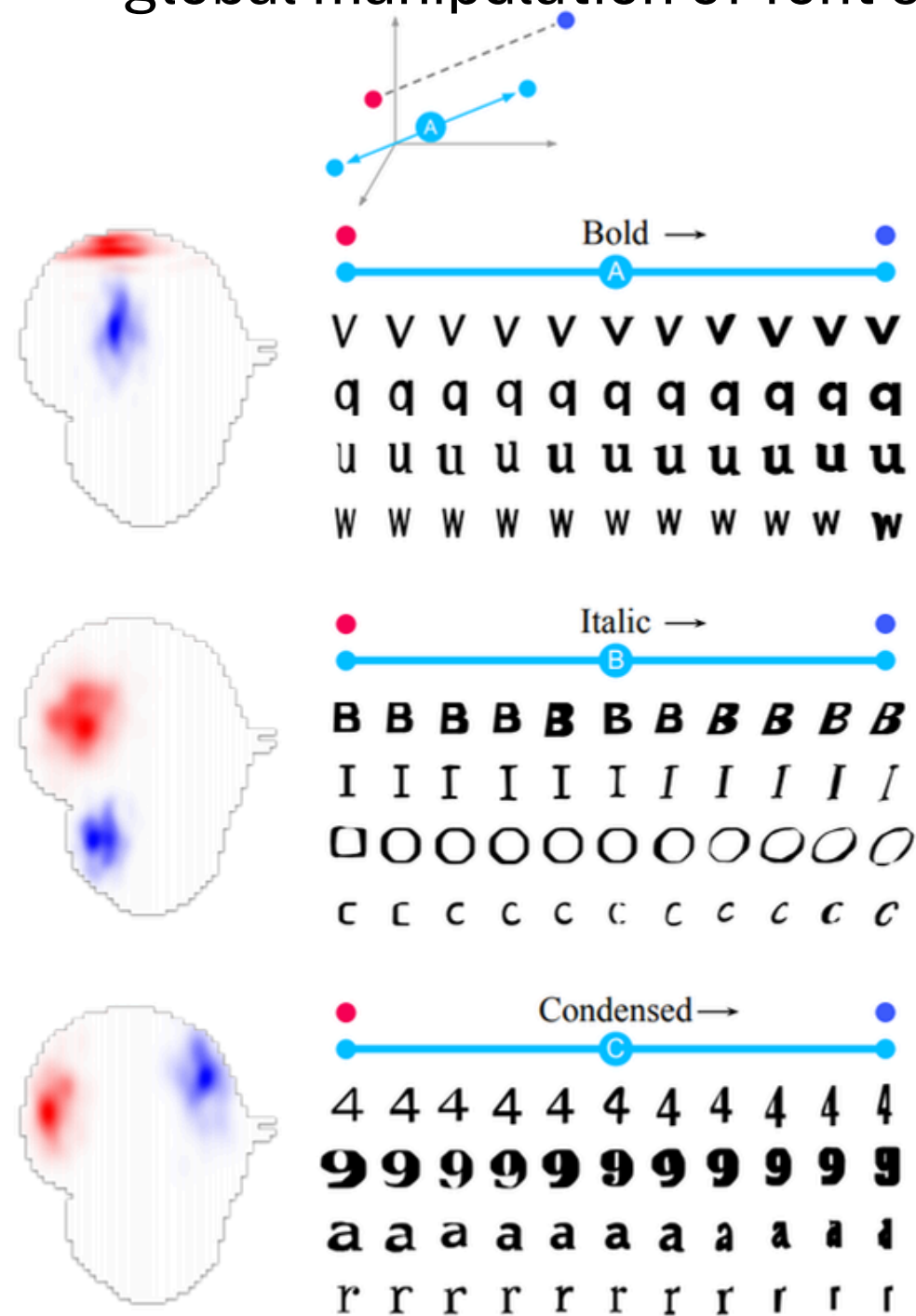
→ z 의 variance 측정 시 더 많은 문자에 의해 조건화 되었을 때 생성된 스타일의 variance가 작아짐

Analysis of representational ability

Building **style analogies** with the learned representation

Q. latent space 내 semantically meaningful direction을 찾을 수 있는가?

→ global manipulation of font style이 가능한가?



- font를 구성하는 semantic concept의 positive/negative 예제를 선택
- concept의 presence와 absence에 상응하는 latent space에서의 영역 확인 (blue : presence, red : absence)
- 평균 z_{red} , z_{blue} 계산, concept direction $c = z_b - z_r$
- $z^* + ac$ (특정 z 에 direction을 더하거나 빼줌)

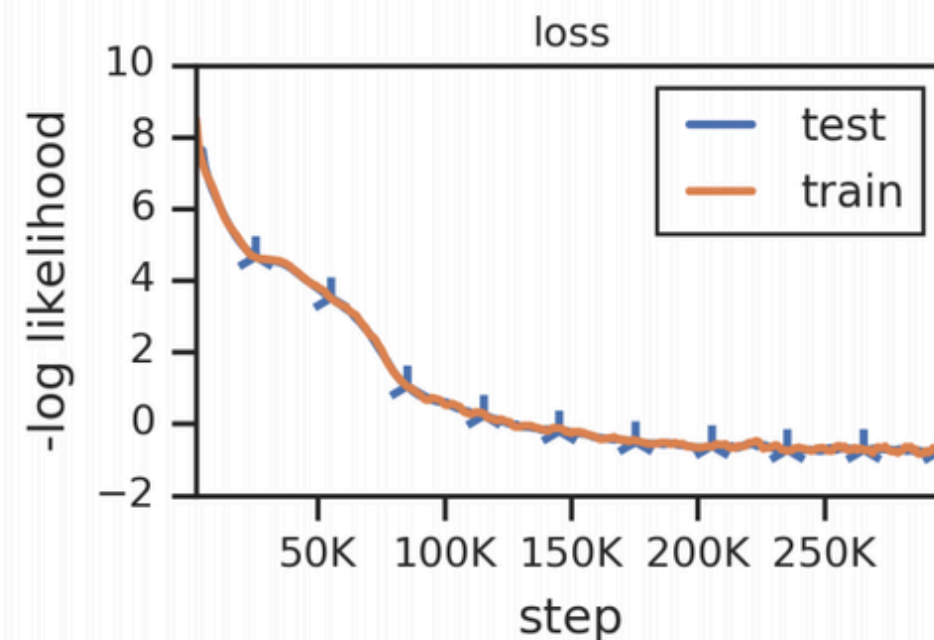
→ modeld concept의 방향에서의 smooth interpolation 관찰 가능
(V '원 → 오' 더 Bold해짐)

- => latent space내의 semantically meaningful direction 해석 가능
- => globally manipulate font style위한 direction찾기도 가능

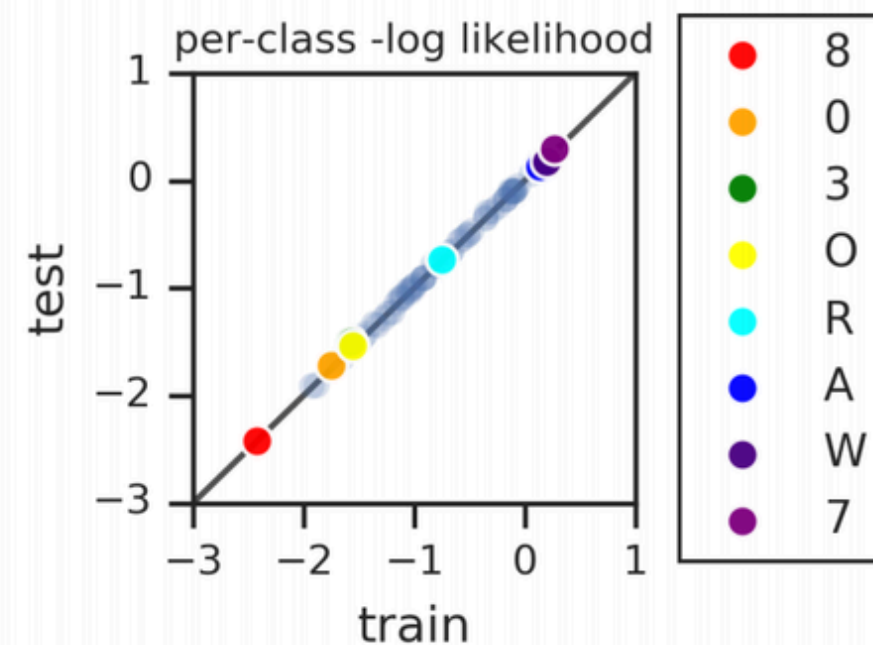
Analysis of representational ability

Quantifying the **quality** of the learned representation

Assessing Generalization



- training dynamics of the model
→ likelihood 계속 증가하다가 평탄해짐
→ overfit X

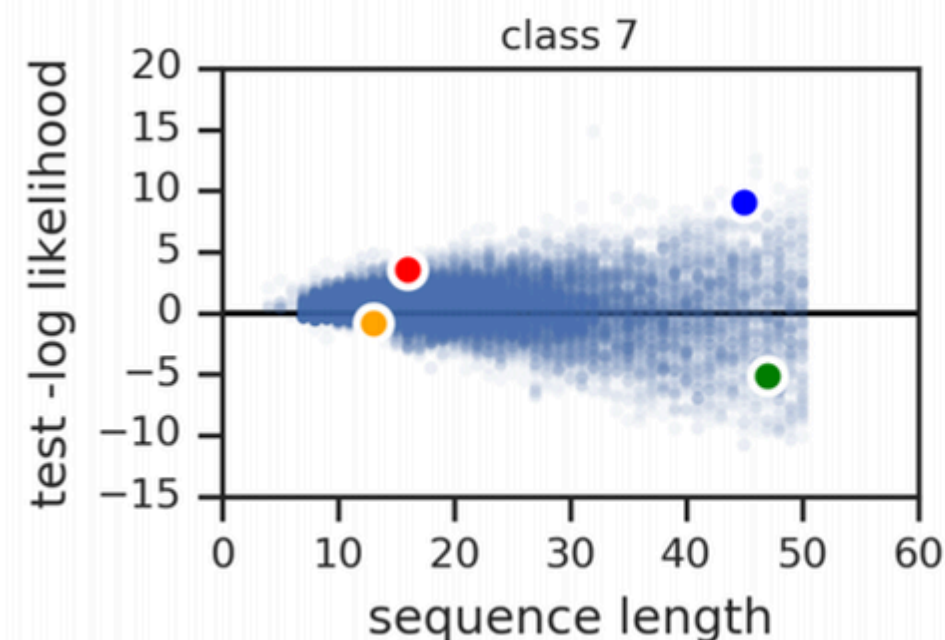
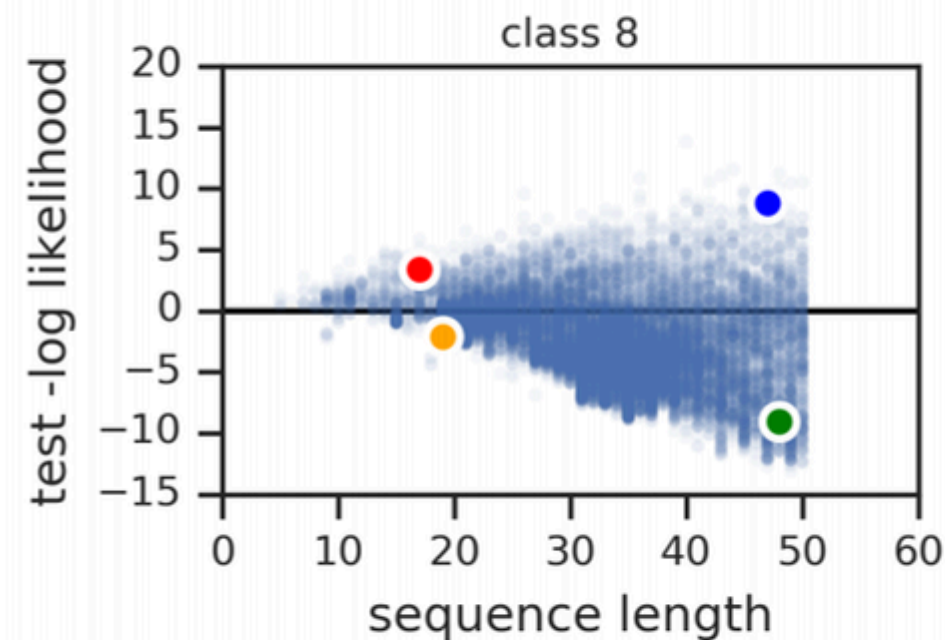


- 각 class 별 평균 negative log likelihood
→ train 성능과 test 성능이 비슷함

Analysis of representational ability

Quantifying the **quality** of the learned representation

Effect of Sequence Length on Decoding Quality



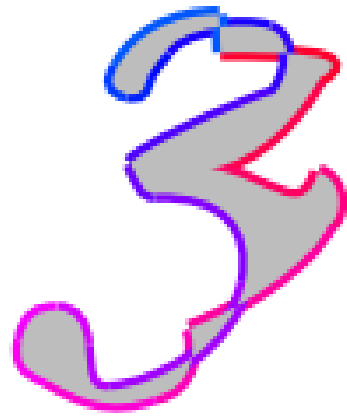
→ sequence 길이가 길수록 log likelihood의 variance가 증가
=> 더 긴 sequence 길이 지닐수록 모델 학습이 안정적X

=> sequence 길이와 상관없이 스타일 복잡성이 높을수록
loss가 높아 학습이 어려움 (red, blue)

Analysis of representational ability

Limitations of working with a learned, stochastic, sequential representation

1. 각 stochastic sampling step에서 모델은 low likelihood decision을 선택함



→ 초기의 low likelihood sample이 수정하기 어려운 연쇄적 error를 초래 (blue → red)



→ time step 지나면서 축적된 error에 의해 시작점과 끝점이 끊김

되돌릴 수 없는 순차적 생성 (초기 선택이 전체 미래를 규정) => error 수정의 teach form으로 교정 가능할 것

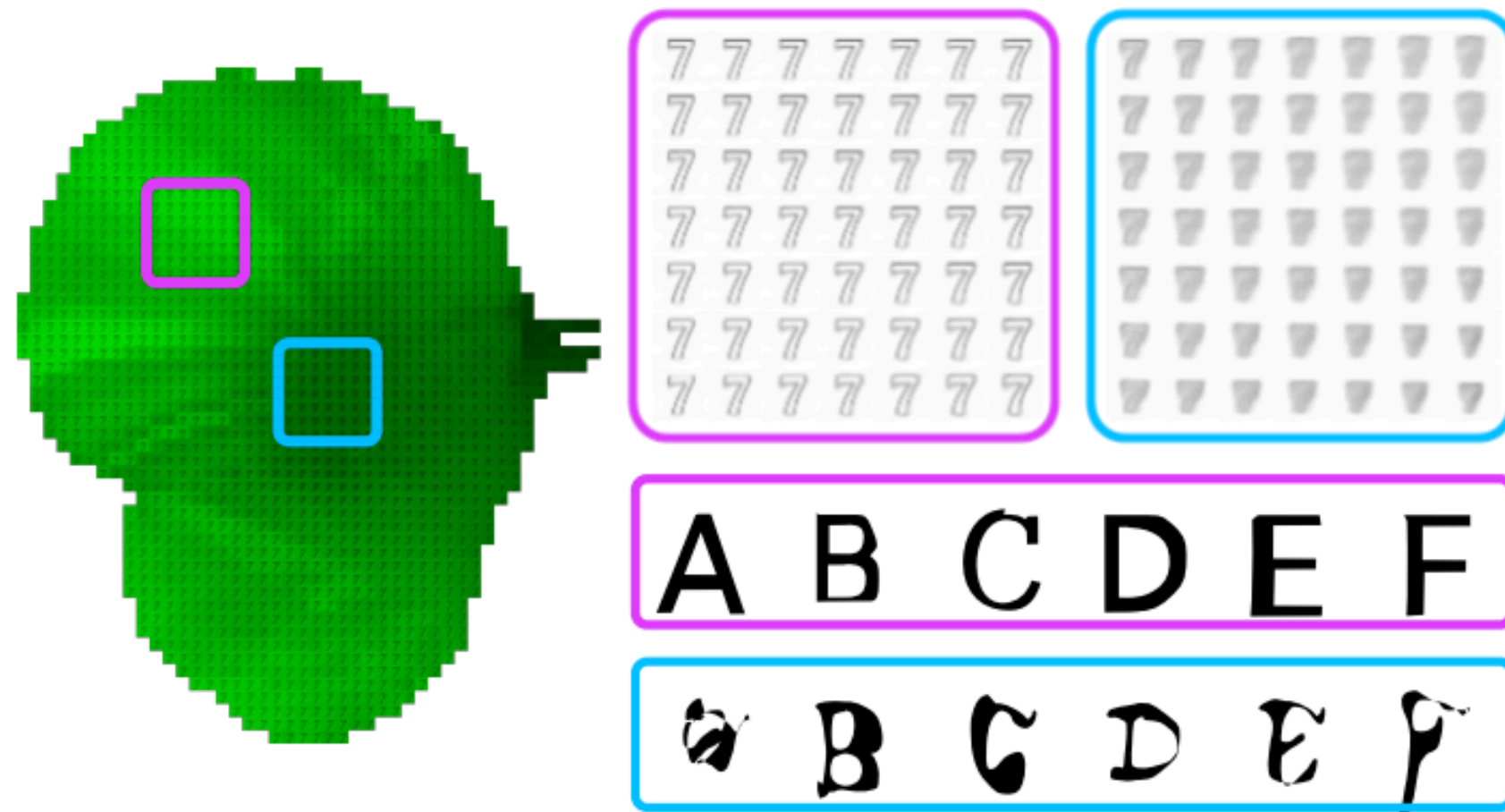
Analysis of representational ability

Limitations of working with a learned, stochastic, sequential representation

2. 모델에서 포착한 uncertainty 반영

→ VAE 잠재 표현의 분산에 의해 모델 예측의 confidence 인지

Quantifying Model Confidence



- green color가 열을 수록 model confidence증가, VAE 분산 감소, sharp output, quality 증가
- green color가 진할 수록 model confidence감소, label noise 증가, more stylized, quality 감소

=> model이 불확실성을 인지하나 해결X

→ 향후 과제

→ visualize the confidence by colorcoding
the UMAP representation for the latent style z by σ_{32}^2

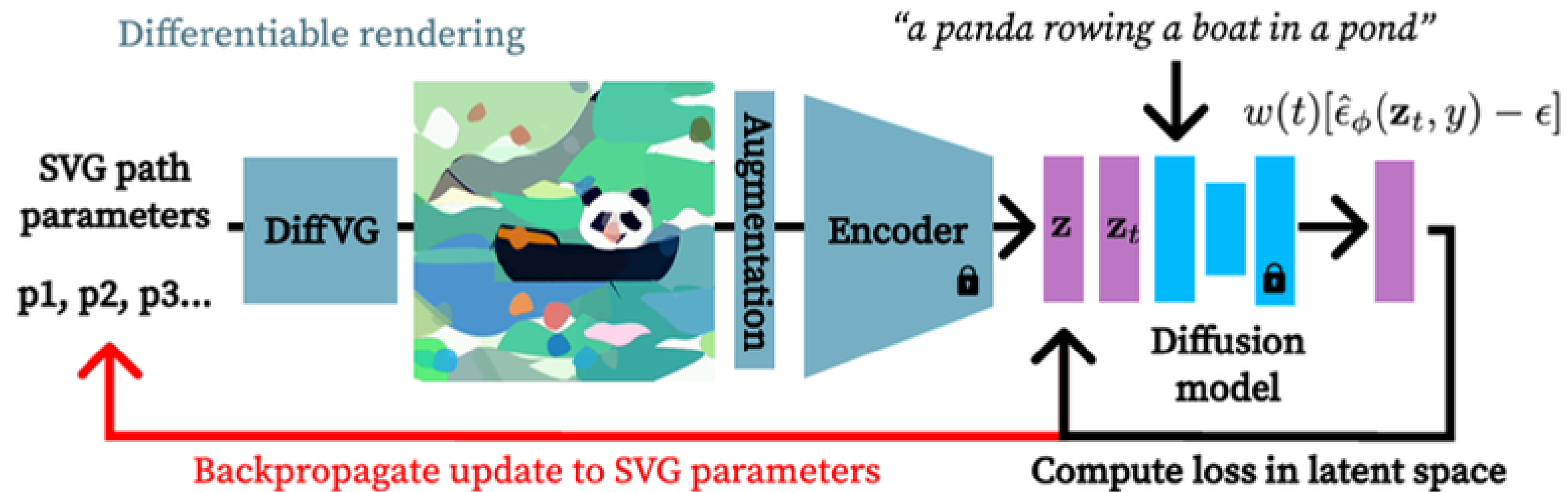
Discussion

- scale-invariant representation 제공
- latent representation 시스템적으로 조작가능, style propagation 수행에 활용 가능
- statistical dependncy와 dataset의 다양성을 담아내기에 sequential, stochastic model의 한계가 존재했음
- time -efficieint 방식으로 인간 디자인 fonts를 돕는 agent로 활용 가능

더 나은 SVG 모델을 위해

- 새로운 attention-based 구조 adversarial-training 도입
- 다른 SVG datasets 활용

	pg.
I What is SVG?	3
II SVG_VAE	8
III Current Trends in SVG Generation Models	9



VectorFusion

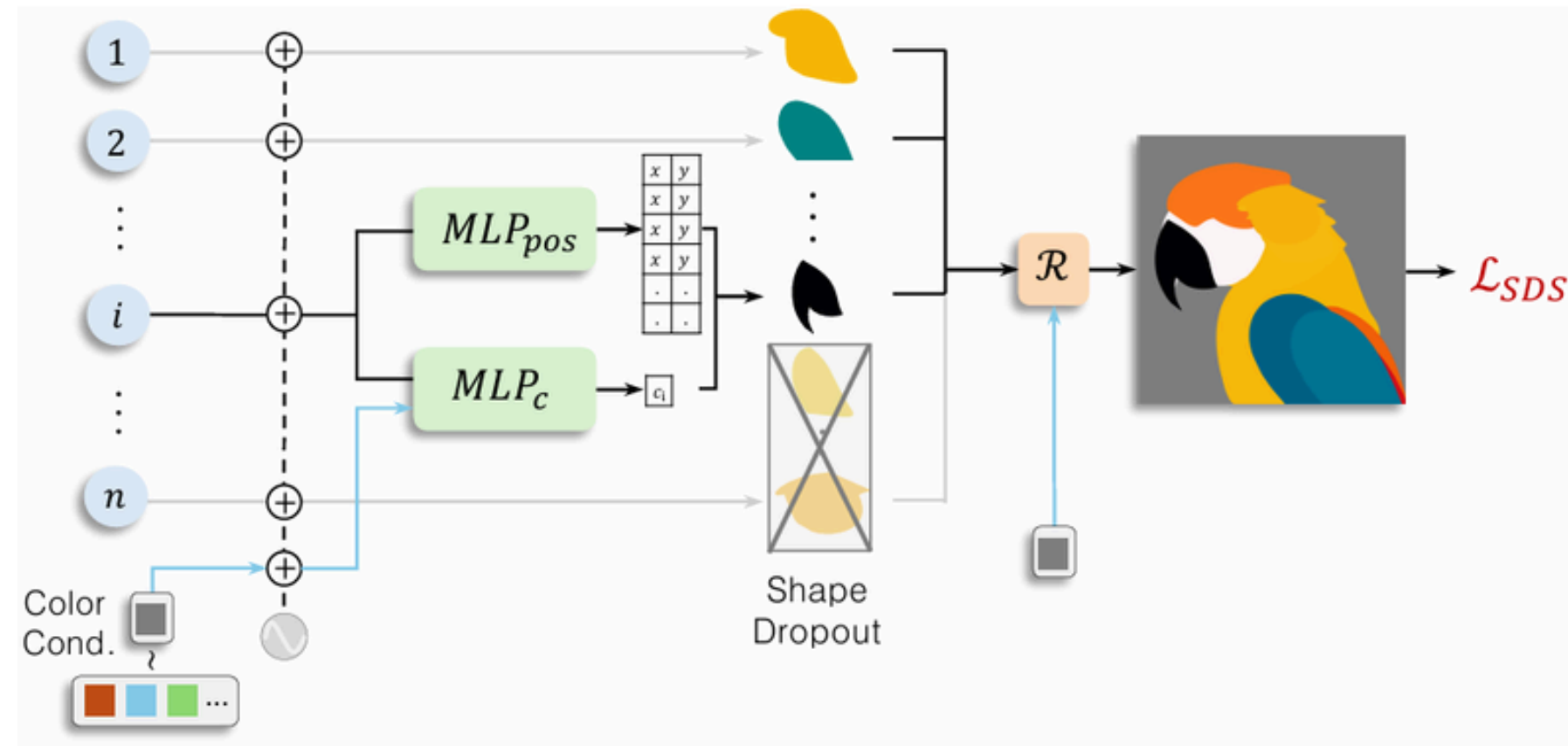
- SVG를 생성하지 않고 SVG 파라미터를 확산 모델의 score에 맞게 반복적으로 최적화
- 확률적 생성X, 연속 최적화
- 전체 SVG를 동시에 보고 반복 수정

NeuralSVG

- SVG 전체를 작은 MLP 가중치로 인코딩
- “도형 i를 물어보면 → MLP가 그 도형을 출력”
- 구조적/계층적 SVG

SVGen

- SVG를 이미지/기하가 아닌 코드 생성 문제로 취급
- LLM이 SVG 문법 자체를 출력
- 잠재공간 샘플링X
- 코드 직접 생성



VectorFusion

- SVG를 생성하지 않고 SVG 파라미터를 확산 모델의 score에 맞게 반복적으로 최적화
- 확률적 생성X, 연속 최적화

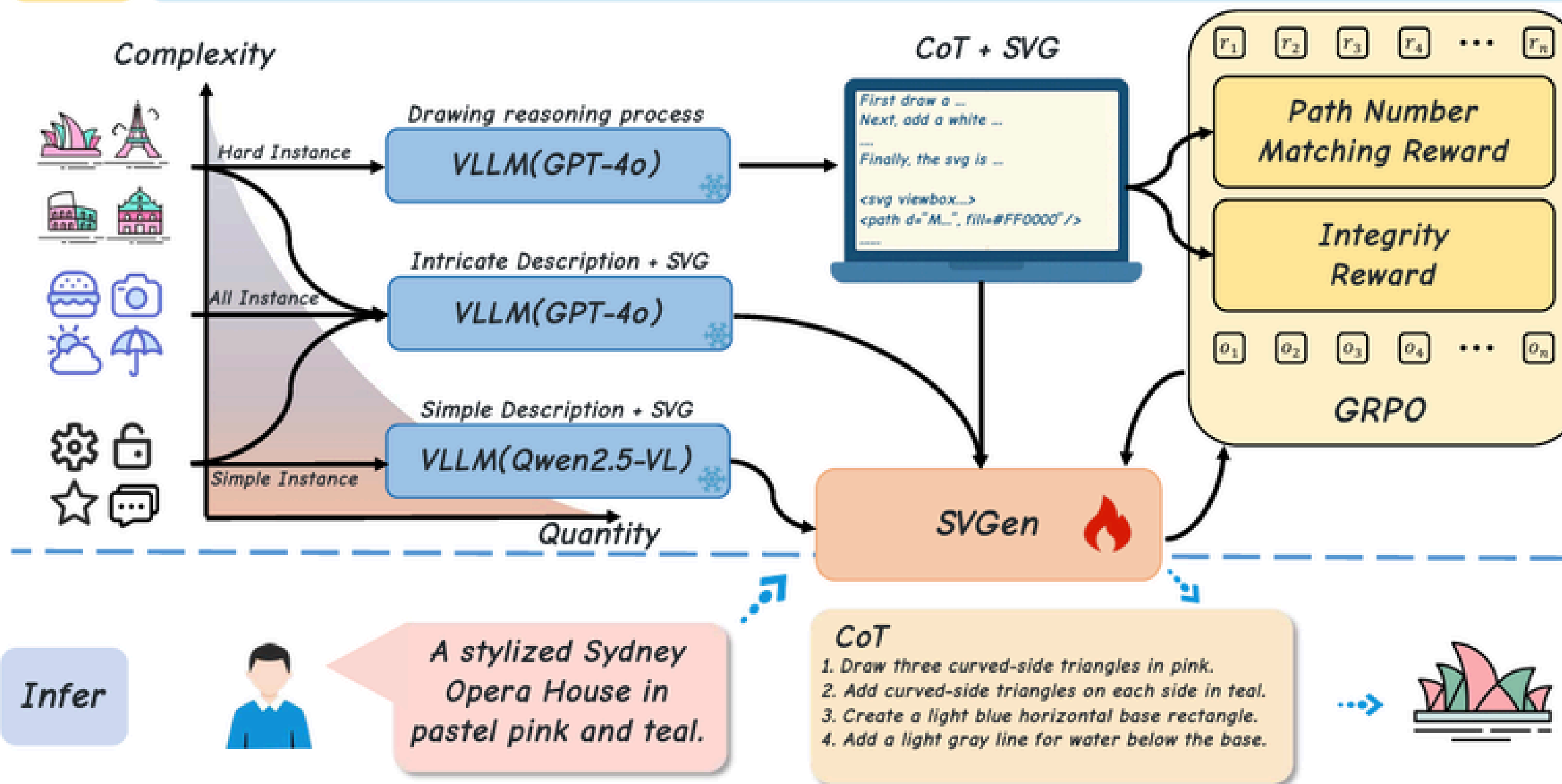
NeuralSVG

- SVG 전체를 작은 MLP 가중치로 인코딩
- “도형 i를 물어보면 → MLP가 그 도형을 출력”
- 구조적/계층적 SVG
- 불확실성을 ‘계층화’로 해결

SVGen

- SVG를 이미지/기하가 아닌 코드 생성 문제로 취급
- LLM이 SVG 문법 자체를 출력
- 잠재공간 샘플링X
- 코드 직접 생성

Train 1: Curriculum Learning 2: Chain-of-thought Learning 3: Reinforcement Learning



SVGen

- SVG를 이미지/기하가 아닌 코드 생성 문제로 취급
 - LLM이 SVG 문법 자체를 출력
 - 잠재공간 샘플링X
 - 코드 직접 생성
- 확률적 생성 자체 제거

Thank You For Listening

STUDENT

Won Jiwoo

AUTHOR

Raphael Gontijo
Lopes etal